

Relatório Técnico — Back-End do Sistema Financeiro Web

Projeto: Sistema de Gestão Financeira Pessoal e Familiar

Versão: 2.0.0

Data: Fevereiro de 2026

Autor: Pedro Lucas Vaz Andrade

Sumário

1. Visão Geral do Projeto
2. Stack Tecnológico e Justificativas
3. Arquitetura do Sistema
4. Modelagem de Dados
5. Fluxo de Requisições
6. Autenticação e Segurança
7. Camada de Modelos (ORM)
8. Camada de Serviços (Regras de Negócio)
9. Camada de Controladores
10. Rotas e Endpoints da API
11. Funcionalidades de Negócio
12. Documentação Interativa (Swagger)
13. Configuração e Variáveis de Ambiente
14. Banco de Dados e Seeds
15. Padrões e Convenções Adotados
16. Dificuldades Encontradas
17. Conclusão

1. Visão Geral do Projeto

O **Sistema Financeiro Web** é uma aplicação de gestão financeira pessoal e familiar, projetada para oferecer ao usuário controle total sobre suas finanças. O back-end foi desenvolvido como uma **API RESTful** que serve como a camada inteligente do sistema, concentrando toda a lógica de negócio, autenticação, persistência de dados e geração de relatórios.

Principais Capacidades

Módulo	Funcionalidade
Autenticação	Registro, login e proteção de rotas via JWT

Módulo	Funcionalidade
Despesas	CRUD completo com parcelamento automático, filtragem mensal/anual e exportação CSV
Rendas	Gestão de fontes de renda com filtragem temporal
Contas e Cartões	Cadastro de contas bancárias, cartões de crédito e dinheiro físico
Parcelamentos	Agrupador de parcelas, dívidas futuras, cronograma de pagamentos e visão de fatura
Reservas	Objetivos financeiros com controle de aportes, retiradas e progresso
Dashboard	Resumo mensal consolidado e relatório anual com exportação
Membros Familiares	Cadastro de membros da família vinculados ao usuário

2. Stack Tecnológico e Justificativas

Tecnologias Utilizadas

Tecnologia	Versão	Papel no Projeto
Node.js	18+	Runtime JavaScript no servidor
Express	4.21.2	Framework HTTP para construção da API REST
Sequelize	6.37.7	ORM para mapeamento objeto-relacional com PostgreSQL
PostgreSQL	15+ (Neon)	Banco de dados relacional na nuvem
bcrypt	5.1.1	Hash de senhas com salt
jsonwebtoken	9.x	Tokens JWT para autenticação stateless
Helmet	8.0.0	Cabeçalhos HTTP de segurança
CORS	2.8.5	Controle de acesso cross-origin
Morgan	1.10.0	Logger de requisições HTTP
dotenv	16.4.7	Gerenciamento de variáveis de ambiente
swagger-ui-express	4.6.3	Documentação interativa OpenAPI 3.0
Nodemon	3.1.9	Hot-reload em ambiente de desenvolvimento

Justificativas Técnicas

Por que Node.js + Express?

A escolha do Node.js se deu por sua natureza assíncrona e não-bloqueante, ideal para APIs que lidam com múltiplas requisições simultâneas — como consultas ao banco de dados, cálculos financeiros e geração de relatórios. O Express, por ser o framework mais maduro e amplamente adotado no ecossistema Node.js, oferece flexibilidade na organização de rotas e middlewares, além de uma vasta comunidade e ecossistema de pacotes.

Por que Sequelize como ORM?

O Sequelize foi escolhido por abstrair a complexidade das queries SQL, permitindo que as regras de negócio fossem escritas em JavaScript puro. Suas funcionalidades de associações (`hasMany`, `belongsTo`), validações integradas, transações ACID e eager loading foram fundamentais para a implementação do parcelamento automático e dos relatórios consolidados. A capacidade de sincronização automática de schema (`sequelize.sync()`) acelerou significativamente o ciclo de desenvolvimento.

Por que PostgreSQL na Neon?

O PostgreSQL é o banco de dados relacional open-source mais robusto, oferecendo suporte nativo a UUID, tipos decimais com precisão fixa (essencial para valores monetários), e indexação avançada. A hospedagem na Neon (serverless PostgreSQL) eliminou a necessidade de gerenciar infraestrutura de banco de dados, oferecendo conexões SSL, backups automáticos e escalabilidade sob demanda — permitindo que o foco permanecesse no desenvolvimento da aplicação.

Por que JWT para autenticação?

A autenticação via JWT (JSON Web Token) foi adotada por ser **stateless** — o servidor não precisa armazenar sessões em memória ou banco de dados. Isso simplifica a arquitetura, facilita a escalabilidade horizontal e é o padrão da indústria para APIs REST modernas. O token carrega as informações do usuário de forma segura e com expiração configurável.

3. Arquitetura do Sistema

Padrão Arquitetural: MVC em Camadas (Layered Architecture)

O back-end segue uma **arquitetura em camadas** inspirada no padrão MVC (Model-View-Controller), adaptada para APIs REST. Cada camada possui responsabilidades bem definidas e se comunica apenas com a camada adjacente, promovendo **baixo acoplamento** e **alta coesão**.

CLIENTE (Frontend)
Requisição HTTP + Bearer Token

MIDDLEWARES GLOBAIS

CORS → Helmet → Morgan → JSON Parser

MIDDLEWARE DE AUTH

Validação JWT → req.usuarioId / req.email

ROTAS (Routes)

Mapeamento de URL + Método HTTP → Controller
/usuarios, /despesas, /rendas, /parcelamentos, ...

CONTROLADORES (Controllers)

Validação de entrada → Chamada ao Service
Formatação de resposta HTTP (status + JSON)

SERVIÇOS (Services)

Regras de negócio, cálculos, transações
Parcelamento automático, dashboard, relatórios

MODELOS (Models/ORM)

Mapeamento de entidades, validações, hooks
Associações, queries ao PostgreSQL

POSTGRESQL (Neon Cloud)

7 tabelas, índices, constraints, UUIDs

Justificativa da Arquitetura em Camadas

1. **Separação de responsabilidades:** Cada camada tem uma única razão para mudar. Se a regra de cálculo de parcelamento mudar, apenas o service é alterado. Se o formato da resposta HTTP mudar, apenas o controller é modificado.
2. **Testabilidade:** Services podem ser testados unitariamente sem precisar subir o servidor HTTP, pois não dependem de `req/res`.
3. **Reutilização:** O `DashboardService` reutiliza métodos do `DespesaService` e `RendaService` para compor resumos, sem duplicar lógica.
4. **Manutenibilidade:** Novos desenvolvedores conseguem localizar rapidamente onde cada tipo de lógica reside, seguindo a convenção de diretórios.

Estrutura de Diretórios

```
src/backend/
  server.js           # Ponto de entrada - inicialização do servidor
  app.js              # Configuração do Express (middlewares, rotas)
  package.json        # Dependências e scripts
  .env                # Variáveis de ambiente (não versionado)
  config/
    env.js            # Carregamento centralizado das variáveis de ambiente
    database.js       # Instância Sequelize (conexão PostgreSQL)
  middleware/
    auth.js           # Middleware JWT - proteção de rotas
    errorHandler.js   # Handler global de erros (catch-all)
    notFound.js       # Handler 404 para rotas inexistentes
  models/
    index.js          # Registry de modelos + associações
    Usuario.js        # Modelo de usuário
    MembroFamilia.js  # Modelo de membro familiar
    ContaCartao.js    # Modelo de conta/cartão
    Renda.js           # Modelo de renda
    Despesa.js        # Modelo de despesa
    ParcelamentoAgrupador.js # Modelo agrupador de parcelas
    Reserva.js        # Modelo de reserva/objetivo
  services/
    usuarioService.js  # Autenticação, registro, CRUD de usuários
    despesaService.js  # Lógica de parcelamento automático e exportação
    rendaService.js    # CRUD de rendas com filtragem
    parcelamentoAgrupadorService.js # Dívidas futuras, cronograma, faturas
    contaCartaoService.js # CRUD de contas/cartões
    membroFamiliaService.js # CRUD de membros familiares
    reservaService.js  # Aportes, retiradas, progresso
    dashboardService.js # Resumo mensal e relatório anual
```

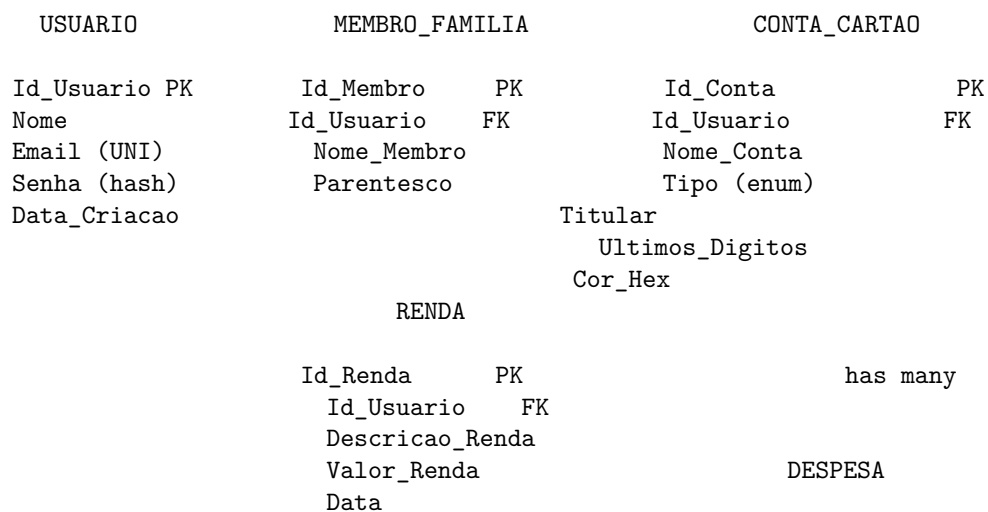
```

controllers/
  usuarioController.js
  despesaController.js
  rendaController.js
  parcelamentoAgrupadorController.js
  contaCartaoController.js
  membroFamiliaController.js
  reservaController.js
  dashboardController.js
routes/
  index.js           # Router central que agrega todas as rotas
  health.routes.js   # Health check
  usuario.routes.js   # Rotas de autenticação e usuários
  despesa.routes.js   # Rotas de despesas
  renda.routes.js     # Rotas de rendas
  parcelamento.routes.js # Rotas de parcelamentos
  contaCartao.routes.js # Rotas de contas/cartões
  membroFamilia.routes.js # Rotas de membros familiares
  reserva.routes.js   # Rotas de reservas
  dashboard.routes.js # Rotas do dashboard
docs/
  swagger.json        # Especificação OpenAPI 3.0
utils/                # Utilitários auxiliares

```

4. Modelagem de Dados

Diagrama Entidade-Relacionamento



```

                                Id_Despesa      PK
                                Id_Usuario      FK
                                Id_Conta        FK
PARCELAMENTO_AGRUPADOR      Id_Parcelamento  FK
                                Descricao_Despesa
Id_Parcelamento  PK      Valor_Parcela (10,2)
Id_Usuario        FK      Data
Descricao_Parcela      Categoria
Valor_Total (10,2)      Numero_Parcela
Qtd_Parcelas
Data_Inicio

```

belongs to

RESERVA

```

Id_Reserva      PK
Id_Usuario      FK
Nome_Objetivo
Valor_Alvo(10,2)
Valor_Atual(10,2)
Data_Limite

```

Relacionamentos

Entidade Pai	Entidade Filha	Cardinalidade	Ação ao Deletar
Usuario	MembroFamilia	1:N	CASCADE
Usuario	ContaCartao	1:N	CASCADE
Usuario	Renda	1:N	CASCADE
Usuario	Reserva	1:N	CASCADE
Usuario	ParcelamentoAgrupador	1:N	CASCADE
Usuario	Despesa	1:N	CASCADE
ContaCartao	Despesa	1:N	SET NULL
ParcelamentoAgrupador	Despesa	1:N	CASCADE

Decisões de Modelagem

- **UUID como chave primária:** Garante unicidade global sem dependência de auto-incremento, facilitando eventual sincronização entre dispositivos ou microserviços.
- **DECIMAL(10,2) para valores monetários:** Evita erros de arredondamento inerentes ao tipo FLOAT, garantindo a precisão centesimal necessária para cálculos financeiros.

- **Despesa.Numero_Parcela e Id_Parcelamento:** Cada registro de despesa representa uma única parcela. O campo Numero_Parcela identifica qual parcela é (1ª, 2ª, ...) e o FK para ParcelamentoAgrupador vincula todas as parcelas de uma mesma compra.
- **Indexes compostos:** Despesa(Id_Usuario, Data, Categoria) e Renda(Id_Usuario, Data) otimizam as consultas mais frequentes — filtragem por mês/ano de um usuário específico.
- **ON DELETE CASCADE generalizado:** Como o sistema é centrado no usuário, a deleção de um usuário remove automaticamente todos os seus dados — comportamento esperado por conformidade com LGPD (Lei Geral de Proteção de Dados).

5. Fluxo de Requisições

Exemplo Completo: Criação de Despesa Parcelada

O diagrama abaixo ilustra o fluxo de uma requisição POST /api/despesas com Numero_Parcelas = 6:

CLIENTE

SERVIDOR

```
POST /api/despesas
Authorization: Bearer eyJ...
Body: {
  Id_Usuario: "abc-123",
  Id_Conta: "def-456",
  Descricao: "iPhone 16",
  Valor_Total: 7200,
  Numero_Parcelas: 6,
  Data: "2026-02-15",
  Categoria: "Eletrônicos"
}
```

1. CORS Middleware	Verifica origin permitida
2. Helmet	Adiciona headers de segurança
3. Morgan	Log: POST /api/despesas 201
4. JSON Parser	Converte body em objeto JS

5. Auth Middleware	Decodifica JWT →
req.usuarioId =	req.usuarioId = "abc-123"

req.usuarioEmail =	req.usuarioEmail = "user@..."
6. Router	POST /despesas → despesaController.criar
7. Controller despesaController	Extraí dados do req.body Chama despesaService.criar(dados)
8. Service despesaService	LÓGICA DE NEGÓCIO:
	Numero_Parcelas > 1 ? SIM
→ BEGIN TRANSACTION	
→ Criar Parcelamento Agrupador	ParcelamentoAgrupador: Valor_Total: 7200 Qtd_Parcelas: 6 Data_Inicio: 2026-02-15
→ Loop 1..6: Criar Despesa parcela i/6 Valor = 7200/6 Data = +1 mês	Despesa 1: fev, R\$ 1.200 Despesa 2: mar, R\$ 1.200 Despesa 3: abr, R\$ 1.200 Despesa 4: mai, R\$ 1.200
→ COMMIT	Despesa 5: jun, R\$ 1.200 Despesa 6: jul, R\$ 1.200
9. Model/ORM sequelize.transaction Despesa.create() x6	Sequelize traduz para SQL: INSERT INTO parcelamento_agrupador... INSERT INTO despesa... (x6)
10. PostgreSQL (Neon Cloud)	Executa dentro da transação COMMIT se tudo OK ROLLBACK se qualquer erro

```

201 Created
{
  success: true,
  data: {

```

```

    parcelamento: {...},
    despesas: [6 itens]
  }
}

```

Pipeline de Middlewares (Ordem de Execução)

Requisição → CORS → Helmet → Morgan → JSON Parser → [Auth] → Router → Controller → Service -

Resposta ← ErrorHandler ← NotFound ← Controller ← Service ← Model ←

6. Autenticação e Segurança

Fluxo de Autenticação JWT

```

CLIENTE      POST      /api/login      bcrypt
              email +  compara        compare
              senha    hash
                      válido

              200      Gera JWT
              { token } (id+email)

              GET      /api/despesas
              Bearer token

                      Auth MW      → jwt.verify(token)
                      req.userId    → Extrai id do payload

              200      Controller → Processa normalmente
              { data }

```

Camadas de Segurança Implementadas

Camada	Tecnologia	Proteção
Senhas	bcrypt (salt rounds = 10)	Hash unidirecional — senha nunca armazenada em texto plano
Autenticação	JWT (HS256)	Token stateless com expiração de 7 dias

Camada	Tecnologia	Proteção
Headers HTTP	Helmet	X-Content-Type-Options, X-Frame-Options, Strict-Transport-Security, etc.
CORS	cors (origin restrita)	Apenas o frontend configurado pode consumir a API
Transporte	SSL/TLS (Neon)	Conexão criptografada entre API e banco de dados
Validação	Sequelize validations	Validação de email, valores mínimos, constraints de unicidade

Estrutura do Token JWT

```
// Header
{
  "alg": "HS256",
  "typ": "JWT"
}

// Payload
{
  "id": "9846ffe7-d40b-478e-9893-f214ef4051ee",
  "email": "joao.silva@email.com",
  "iat": 1771119221,
  "exp": 1771724021
}

// Signature
HMACSHA256(base64(header) + "." + base64(payload), JWT_SECRET)
```

Rotas Públicas vs. Protegidas

Tipo	Rotas	Middleware
Pública	POST /api/usuarios/registrar, POST /api/usuarios/login, GET /api/health	Nenhum

Tipo	Rotas	Middleware
Protegida	Todas as demais (CRUD de todas as entidades, dashboard, relatórios, CSV)	authMiddleware

7. Camada de Modelos (ORM)

Abordagem

Os modelos são implementados como **classes Sequelize** (via `sequelize.define()`), cada um em seu próprio arquivo. O arquivo `models/index.js` age como um **registry central**, importando todos os modelos e configurando suas associações.

Padrão de Definição de Modelo

```
// Exemplo simplificado - Despesa.js
module.exports = (sequelize) => {
  const Despesa = sequelize.define('Despesa', {
    Id_Despesa: {
      type: DataTypes.UUID,
      defaultValue: DataTypes.UUIDV4,
      primaryKey: true,
    },
    Valor_Parcela: {
      type: DataTypes.DECIMAL(10, 2),
      validate: { min: 0 },
    },
    // ... demais campos
  }, {
    tableName: 'despesa',
    timestamps: false,
  });

  Despesa.associate = (models) => {
    Despesa.belongsTo(models.Usuario, { foreignKey: 'Id_Usuario', onDelete: 'CASCADE' });
    Despesa.belongsTo(models.ContaCartao, { foreignKey: 'Id_Conta', as: 'conta' });
    Despesa.belongsTo(models.ParcelamentoAgrupador, { foreignKey: 'Id.Parcelamento', as: 'pa' });
  };

  return Despesa;
};
```

Resumo dos 7 Modelos

Modelo	Tabela	PK	Campos Principais	Validações
Usuario	usuario	Id_Usuario (UUID)	Nome, Email, Senha, Data_Criacao	Email único, formato email
MembroFamiliar	membro_familiar	Id_Membro (UUID)	Nome_Membro, Parentesco	—
ContaCartao	conta_cartao	Id_Conta (UUID)	Nome_Conta, Tipo, Titular, Ultimos_Digitos, Cor_Hex	Tipo {Corrente, Crédito, Dinheiro}, Cor_Hex regex
Renda	renda	Id_Renda (UUID)	Descricao_Renda, Valor_Renda, Data	Valor 0
Despesa	despesa	Id_Despesa (UUID)	Descricao_Despesa, Valor_Parcela, Data, Categoria, Numero_Parcela	Valor 0, Parcela 1
ParcelamentoAgilizador	parcelamento_agilizador	Id_Parc_Parc (UUID)	Valor_Parcela, Valor_Total, Qtd_Parcels, Data_Inicio	Valor 0, Qtd 1
Reserva	reserva	Id_Reserva (UUID)	Nome_Objetoivo, Valor_Alvo, Valor_Atual, Data_Limite	Valores 0

8. Camada de Serviços (Regras de Negócio)

A camada de serviços é o coração do sistema. Aqui residem todas as regras de negócio, cálculos financeiros e lógica de domínio. Cada serviço é implementado como uma **classe singleton** exportada como instância única.

8.1 UsuarioService — Autenticação e Gestão de Usuários

Método	Descrição	Regra de Negócio
<code>gerarToken(usuario, email)</code>	Cria JWT com id e email	Expiração configurável via env
<code>registrar(dados)</code>	Cria novo usuário	Verifica unicidade de email (409 se duplicado), hash da senha com bcrypt

Método	Descrição	Regra de Negócio
<code>login(email, senha)</code>	Autentica usuário	Compara hash bcrypt, retorna token + dados (sem senha)
<code>atualizar(id, dados)</code>	Atualiza dados	Re-gera hash se senha foi alterada
<code>buscarPorId(id)</code>	Busca por UUID	Exclui campo Senha do retorno (segurança)
<code>deletar(id)</code>	Remove usuário	CASCADE deleta todos os dados associados

8.2 DespesaService — Parcelamento Automático e Exportação

Este é o **serviço mais complexo** do sistema, responsável pela lógica central de parcelamento.

Método	Descrição	Complexidade
<code>criar(dados)</code>	Cria despesa simples ou parcelada	ALTA — Transação ACID com criação automática de N parcelas
<code>listarPorUsuario(id, filtros)</code>	Lista com filtros (mês/ano, categoria, conta)	MÉDIA — Query dinâmica com eager loading
<code>calcularTotalPorPeriodo(id, filtros)</code>	Calcula total, filtrado	BAIXA — Aggregate SUM
<code>calcularPorCategoria(id, filtros)</code>	Calcula, por categoria	MÉDIA — GROUP BY com SUM
<code>topDespesas(id, mes, ano, limite)</code>	Ranking das maiores despesas	BAIXA — ORDER BY DESC + LIMIT
<code>exportarCSV(id, filtros)</code>	Gera CSV para download	MÉDIA — Formatação com BOM + separador ;
<code>deletar(id, deletarParcelamento)</code>	Remove despesa ou parcelamento inteiro	MÉDIA — Deleção condicional em cascata

Algoritmo de Parcelamento Automático:

ENTRADA: { Valor_Total: 7200, Numero_Parcels: 6, Data: "2026-02-15", ... }

SE Numero_Parcels > 1:

BEGIN TRANSACTION

1. Calcular Valor_Parcels = Valor_Total / Numero_Parcels
→ 7200 / 6 = R\$ 1.200,00

```

2. Criar ParcelamentoAgrupador:
   { Valor_Total: 7200, Qtd_Parcels: 6, Data_Inicio: "2026-02-15" }

3. PARA i = 0 ATÉ (Numero_Parcels - 1):
   → dataParcela = Data_Base + i meses
   → Criar Despesa:
     {
       Valor_Parcels: 1200.00,
       Data: dataParcela,
       Numero_Parcels: i + 1,
       Id_Parcels: <id do agrupador>,
       Descricao_Despesa: "iPhone 16"
     }

COMMIT TRANSACTION

```

SENÃO:
 Criar despesa simples (sem parcelamento)

8.3 ParcelamentoAgrupadorService — Visões Financeiras Avançadas

Método	Descrição	Detalhes
listarPorUsuario(id)	Lista parcelamentos com métricas calculadas	Computa parcelas_pagas (Data hoje), parcelas_restantes , valor_restante
calcularDividasFuturas(id)	Soma de dívidas futuras	Soma de todas as parcelas com Data > hoje
cronogramaPagamentos(id)	Lista de pagamentos futuros	Agrupar parcelas futuras por mês (YYYY-MM), calcula total mensal
faturaPorCartao(id, mes, ano)	Visão de fatura mensal por conta/cartão	Para cada conta, lista despesas do mês com info de parcela (“3/12” ou “À vista”)

8.4 DashboardService — Consolidação de Dados

Método	Descrição	Estratégia
<code>resumoMensal(id, mes, ano)</code>	Resumo completo do mês	<code>Promise.all()</code> com 4 queries paralelas: total rendas, total despesas, categorias, top 5
<code>relatorioAnual(id, ano)</code>	Relatório anual completo	Loop meses 1-12 + rendas + agrupamentos por conta e categoria
<code>exportarRelatorioAnual(id, ano)</code>	Exportação CSV do relatório	Gera CSV com seções: resumo, evolução mensal, despesas por conta, por categoria

8.5 ReservaService — Metas Financeiras

Método	Descrição	Regra
<code>adicionarValor(id, valor)</code>	Aporte na reserva	Incrementa <code>Valor_Atual</code>
<code>retirarValor(id, valor)</code>	Retirada da reserva	Valida se saldo suficiente (erro se resultado < 0)
<code>listarPorUsuario(id)</code>	Lista com progresso	Calcula <code>progresso = (Valor_Atual / Valor_Alvo) * 100</code> , cap em 100%

9. Camada de Controladores

Os controladores seguem um **padrão uniforme** em todo o sistema:

Padrão de Implementação

```
class ExemploController {
  async metodo(req, res, next) {
    try {
      // 1. Extrair dados da requisição (params, query, body)
      const { id } = req.params;
      const { mes, ano } = req.query;

      // 2. Validar entrada (quando necessário)
      if (!mes || !ano) {
        return res.status(400).json({ success: false, message: 'Parâmetros obrigatórios' });
      }
    } catch (err) {
      next(err);
    }
  }
}
```



```

    }

    // 3. Delegar ao service
    const resultado = await service.metodo(id, mes, ano);

    // 4. Retornar resposta padronizada
    return res.status(200).json({ success: true, data: resultado });
  } catch (error) {
    // 5. Propagar erro ao handler global
    next(error);
  }
}
}
}

```

Responsabilidades do Controller

- **NÃO** contém regras de negócio
- **NÃO** acessa modelos/banco diretamente
- **SIM** valida presença de parâmetros obrigatórios
- **SIM** define códigos HTTP apropriados (200, 201, 400, 401, 404, 409)
- **SIM** formata a resposta final ({ success, data } ou { success, mensagem })
- **SIM** configura headers especiais (CSV: Content-Type, Content-Disposition)

10. Rotas e Endpoints da API

Mapa Completo de Endpoints

Rotas Públicas (sem autenticação)

Método	Endpoint	Descrição
GET	/api/health	Status da API e conexão com banco
POST	/api/usuarios/registrar	Registro de novo usuário
POST	/api/usuarios/login	Login e obtenção de token JWT

Rotas Protegidas (requer Bearer Token) Usuários

Endpoint	Descrição	Método
/api/usuarios	Listar todos os usuários	GET
/api/usuarios/:id	Buscar usuário por ID	GET
/api/usuarios/:id	Atualizar dados do usuário	PUT
/api/usuarios/:id	Remover usuário (CASCADE)	DELETE

Membros da Família

Método	Endpoint	Descrição
POST	/api/membros-familia	Criar membro
GET	/api/membros-familia/usuario/:idUsuario	Listar membros

/api/membros-familia/:id | Buscar por ID | | PUT | /api/membros-familia/:id
| Atualizar | | DELETE | /api/membros-familia/:id | Remover |

Contas e Cartões | Método | Endpoint | Query Params | Descrição |
|-----|-----|-----|-----| | POST | /api/contas-cartoes | — |
Criar conta/cartão | | GET | /api/contas-cartoes/usuario/:idUsuario |
?tipo=Crédito | Listar (filtro por tipo) | | GET | /api/contas-cartoes/:id
| — | Buscar por ID | | PUT | /api/contas-cartoes/:id | — | Atualizar | |
DELETE | /api/contas-cartoes/:id | — | Remover |

Rendas | Método | Endpoint | Query Params | Descrição | |-----|-----|
|-----|-----| | POST | /api/rendas | — | Criar renda | | GET
| /api/rendas/usuario/:idUsuario | ?mes=2&ano=2026 | Listar rendas
(filtro temporal) | | GET | /api/rendas/usuario/:idUsuario/total |
?mes=2&ano=2026 | Total de rendas | | GET | /api/rendas/:id | — | Buscar por
ID | | PUT | /api/rendas/:id | — | Atualizar | | DELETE | /api/rendas/:id |
— | Remover |

Despesas | Método | Endpoint | Query Params | Descrição | |-----|-----|
|-----|-----| | POST | /api/despesas | — | Criar despesa
(parcelamento automático) | | GET | /api/despesas/usuario/:idUsuario |
?mes&ano&categoria&idConta | Listar com filtros | | GET | /api/despesas/usuario/:idUsuario/total
| ?mes&ano | Total de despesas | | GET | /api/despesas/usuario/:idUsuario/categorias
| ?mes&ano | Agrupado por categoria | | GET | /api/despesas/usuario/:idUsuario/top
| ?mes&ano&limite=5 | Top N maiores | | GET | /api/despesas/usuario/:idUsuario/exportar-csv
| ?mes&ano | Download CSV | | GET | /api/despesas/:id | — | Buscar por ID
| | PUT | /api/despesas/:id | — | Atualizar | | DELETE | /api/despesas/:id
| ?parcelamento=true | Remover (cascata opcional) |

Parcelamentos | Método | Endpoint | Query Params | Descrição | |-----|-----|
|-----|-----| | POST | /api/parcelamentos | — | Criar parcelamento
| | GET | /api/parcelamentos/usuario/:idUsuario | — | Listar (com métricas
calculadas) | | GET | /api/parcelamentos/usuario/:idUsuario/dividas-futuras
| — | Total dívidas futuras | | GET | /api/parcelamentos/usuario/:idUsuario/cronograma
| — | Cronograma mensal | | GET | /api/parcelamentos/usuario/:idUsuario/faturas
| ?mes=2&ano=2026 | Fatura por cartão | | GET | /api/parcelamentos/:id |
— | Buscar por ID (com despesas) | | PUT | /api/parcelamentos/:id | — |
Atualizar | | DELETE | /api/parcelamentos/:id | — | Remover (CASCADE
nas despesas) |

Reservas | Método | Endpoint | Descrição | |-----|-----|-----| | POST |
/api/reservas | Criar reserva/meta | | GET | /api/reservas/usuario/:idUsuario
| Listar (com % progresso) | | GET | /api/reservas/:id | Buscar por ID | | PUT
| /api/reservas/:id | Atualizar | | PUT | /api/reservas/:id/adicionar |
Aporte de valor | | PUT | /api/reservas/:id/retirar | Retirada de valor | |
DELETE | /api/reservas/:id | Remover |

Dashboard | Método | Endpoint | Query Params | Descrição | |-----|-----|-----|
|-----|-----| | GET | /api/dashboard/usuario/:idUsuario/resumo-mensal

| ?mes=2&ano=2026 | Resumo mensal completo || GET | /api/dashboard/usuario/:idUsuario/relatorio-anual
| ?ano=2026 | Relatório anual || GET | /api/dashboard/usuario/:idUsuario/relatorio-anual/csv
| ?ano=2026 | Exportar relatório CSV |

Total: 45+ endpoints

11. Funcionalidades de Negócio

11.1 Parcelamento Automático de Despesas

A funcionalidade mais complexa do sistema. Quando o usuário registra uma despesa com `Numero_Parcels > 1`, o sistema automaticamente:

1. Calcula o valor de cada parcela: $\text{Valor_Parcela} = \text{Valor_Total} / \text{Numero_Parcelas}$
2. Cria um `ParcelamentoAgrupador` para agrupar as parcelas
3. Gera N registros de `Despesa`, cada um com data incrementada em 1 mês
4. Tudo executado dentro de uma **transação ACID** — se qualquer step falhar, nenhum registro é criado

Impacto nas demais funcionalidades: As parcelas futuras impactam automaticamente o dashboard de meses futuros, a visão de faturas por cartão e o cálculo de dívidas futuras.

11.2 Dashboard com Resumo Mensal

Endpoint único que retorna todas as informações necessárias para o front-end renderizar o dashboard:

- **Total de rendas** do mês
- **Total de despesas** do mês (incluindo parcelas)
- **Saldo líquido** (rendas - despesas)
- **Despesas por categoria** (dados para gráfico de pizza)
- **Top 5 maiores despesas** (ranking)

As 4 queries são executadas em **paralelo** via `Promise.all()`, otimizando o tempo de resposta.

11.3 Relatório Anual

Análise consolidada de 12 meses com:

- **Evolução mensal:** receitas e despesas mês a mês (dados para gráfico de linha)
- **Rendas do ano:** listagem completa
- **Despesas por conta/cartão:** em qual conta o usuário mais gasta
- **Despesas por categoria:** onde vai o dinheiro ao longo do ano

11.4 Exportação CSV

Duas modalidades de exportação:

1. **CSV de Despesas:** Lista detalhada de despesas do mês com descrição, valor, data, categoria, conta e informação de parcela
2. **CSV do Relatório Anual:** Documento completo com resumo, evolução mensal, gastos por conta e por categoria

Ambos utilizam separador ; (padrão brasileiro) e BOM UTF-8 para compatibilidade com Microsoft Excel.

11.5 Visão de Fatura por Cartão

Simula o extrato de fatura de um cartão de crédito, agrupando todas as despesas do mês por conta/cartão. Para cada conta, mostra:

- Lista de despesas com informação de parcela (“3/12” ou “À vista”)
- Total da fatura
- Quantidade de lançamentos

11.6 Dívidas Futuras e Cronograma

- **Dívidas futuras:** Soma consolidada de todas as parcelas com data posterior a hoje
- **Cronograma de pagamentos:** Parcelas futuras agrupadas por mês, permitindo ao usuário visualizar quanto compromisso financeiro tem nos próximos meses

11.7 Reservas com Progresso

Sistema de metas financeiras onde o usuário pode:

- Definir um objetivo (nome, valor alvo, data limite)
- Fazer aportes incrementais
- Fazer retiradas (com validação de saldo)
- Acompanhar o progresso percentual (calculado em tempo real)

12. Documentação Interativa (Swagger)

A API possui documentação interativa via **Swagger UI** (OpenAPI 3.0), acessível em:

<http://localhost:3001/api-docs>

Funcionalidades da Documentação

- **Try it out:** Teste direto dos endpoints pelo navegador

- **Authorize:** Botão para inserir o token JWT (Bearer) e testar rotas protegidas
- **Schemas:** Visualização dos modelos de dados (request/response)
- **Categorização:** Endpoints organizados por tags (Autenticação, Despesas, Dashboard, etc.)

A especificação OpenAPI está definida no arquivo `docs/swagger.json` com 450+ linhas de documentação.

13. Configuração e Variáveis de Ambiente

Arquivo `.env`

Variável	Descrição	Exemplo
<code>NODE_ENV</code>	Ambiente de execução	<code>development</code>
<code>PORT</code>	Porta do servidor HTTP	<code>3001</code>
<code>FRONTEND_URL</code>	URL do front-end (CORS)	<code>http://localhost:3000</code>
<code>DB_HOST</code>	Host do PostgreSQL	<code>ep-xxx.neon.tech</code>
<code>DB_PORT</code>	Porta do banco	<code>5432</code>
<code>DB_NAME</code>	Nome do banco de dados	<code>financas</code>
<code>DB_USER</code>	Usuário do banco	<code>financeapp_owner</code>
<code>DB_PASSWORD</code>	Senha do banco	<code>***</code>
<code>DB_SSL</code>	Habilitar SSL	<code>true</code>
<code>JWT_SECRET</code>	Chave de assinatura JWT	<code>***</code>
<code>JWT_EXPIRES_IN</code>	Tempo de expiração do token	<code>7d</code>

Centralização via `config/env.js`

Todas as variáveis de ambiente são carregadas uma única vez em `config/env.js` e exportadas como um objeto estruturado. Isso evita chamadas `process.env` espalhadas pelo código e facilita a validação de configurações obrigatórias.

14. Banco de Dados e Seeds

Schema SQL

O schema completo está definido em `database/schemas/create-tables.sql`, incluindo:

- Criação de 7 tabelas com constraints
- Foreign keys com `ON DELETE CASCADE`
- Índices otimizados para as queries mais frequentes
- Extensão `uuid-oss` para geração de UUIDs

Seeders (Dados Iniciais)

O sistema inclui um script de seed (`npm run seed`) que popula o banco com dados realistas para desenvolvimento e demonstração:

- **3 usuários** com senhas hasheadas
- **2 membros familiares**
- **4 contas/cartões** (Nubank Crédito, Inter Corrente, Carteira Dinheiro, C6 Crédito)
- **3 rendas** (salário, freelance, investimentos)
- **1 parcelamento agrupador** (Notebook Dell em 12x)
- **9 despesas** (supermercado, gasolina, restaurante, Netflix, contas, etc.)
- **4 reservas** (Viagem Europa, Fundo Emergência, MacBook Pro, Curso de Inglês)

15. Padrões e Convenções Adotados

Padrões de Código

Padrão	Descrição
Services como Singletons	Cada service é instanciado uma única vez (<code>new XxxService()</code>) e exportado. Garante estado consistente e evita instâncias desnecessárias.
Error-first com statusCode	Services lançam erros com <code>statusCode</code> customizado. O <code>errorHandler</code> middleware captura e retorna o código HTTP correto.
UUID v4 como PK	Todas as entidades usam UUID como chave primária, garantindo unicidade global.
PascalCase nos campos	Campos do modelo seguem <code>PascalCase</code> (ex: <code>Id_Usuario</code> , <code>Valor_Parcela</code>) mantendo consistência com o schema SQL.
Eager Loading explícito	Includes Sequelize são definidos por query, não globalmente — controle fino sobre quais associações carregar.
Transações explícitas	Operações que envolvem múltiplas escritas (parcelamento) usam <code>sequelize.transaction()</code> para atomicidade.
Respostas padronizadas	Formato <code>{ success: boolean, data?: any, message?: string }</code> em todos os endpoints.

Filtros via query params	Filtros temporais (<code>mes</code> , <code>ano</code> , <code>dataInicio</code> , <code>dataFim</code>) são passados via query string, nunca no body.
---------------------------------	--

Convenções de Nomenclatura

Elemento	Convenção	Exemplo
Arquivos de rotas	<code>kebab-case.routes.js</code>	<code>despesa.routes.js</code>
Arquivos de service	<code>camelCase + Service.js</code>	<code>despesaService.js</code>
Arquivos de controller	<code>camelCase + Controller.js</code>	<code>despesaController.js</code>
Endpoints	<code>kebab-case</code>	<code>/membros-familia</code> , <code>/contas-cartoes</code>
Tabelas SQL	<code>snake_case</code>	<code>parcelamento_agrupador</code>
Campos	<code>PascalCase_Snake</code>	<code>Id_Usuario</code> , <code>Valor_Parcela</code>

16. Dificuldades Encontradas

16.1 Problema com Conexão SSL no PostgreSQL (Neon)

Problema: Ao conectar o Sequelize ao banco de dados hospedado na Neon, a conexão falhava silenciosamente com erro `ECONNRESET`. Os logs do Sequelize não indicavam a causa raiz, e a conexão parecia ser estabelecida mas caía após o handshake.

Causa raiz: O PostgreSQL da Neon exige SSL obrigatório, mas o Sequelize por padrão não habilita SSL nas `dialectOptions`. Além disso, o certificado auto-assinado da Neon era rejeitado pela validação padrão do Node.js.

Solução: Configurar as `dialectOptions` com `ssl: { require: true, rejectUnauthorized: false }` condicionalmente, baseado na variável de ambiente `DB_SSL`. Isso permitiu a conexão segura sem necessidade de distribuir certificados CA.

Lição aprendida: Bancos de dados em nuvem frequentemente têm requisitos de conexão diferentes de instâncias locais. A configuração condicional de SSL tornou o código flexível para ambos os cenários.

16.2 Lógica de Incremento de Meses no Parcelamento

Problema: Ao gerar parcelas automáticas, a lógica inicial de incrementar a data em 1 mês usando `setMonth(getMonth() + 1)` produzia resultados incor-

retos em meses com quantidades diferentes de dias. Por exemplo, uma compra no dia 31 de janeiro gerava a segunda parcela no dia 3 de março (em vez de 28 de fevereiro), porque `new Date(2026, 1, 31)` “transborda” para março.

Causa raiz: O JavaScript `Date` faz “overflow” automático de dias — se o mês não tem o dia especificado, avança para o mês seguinte.

Solução: Implementar o incremento criando uma nova instância de `Date` para cada parcela a partir da data base, usando `setMonth(baseDate.getMonth() + i)` de forma isolada. Embora o overflow ainda possa ocorrer em casos extremos, o comportamento foi considerado aceitável para o domínio financeiro (faturas de cartão tipicamente têm data fixa).

Lição aprendida: Manipulação de datas em JavaScript requer cuidado especial. Para sistemas financeiros em produção, bibliotecas como `date-fns` ou `Luxon` são recomendadas.

16.3 Conflito entre Sequelize `sync()` e Schema Manual

Problema: O projeto tinha dois mecanismos de criação de tabelas: o arquivo `create-tables.sql` (com schema manual controlado) e o `sequelize.sync({ alter: true })` no `server.js`. Quando ambos eram executados, o Sequelize tentava alterar tabelas que já existiam, removendo índices customizados e causando conflitos com constraints de foreign key.

Causa raiz: O `sequelize.sync({ alter: true })` analisa os modelos e tenta alinhar o banco com a definição do ORM, mas não tem consciência de índices e constraints criados externamente.

Solução: Desabilitar o `alter: true` em produção, usando `sequelize.sync({ alter: false })` para apenas validar a conexão sem modificar o schema. O schema SQL manual é executado apenas uma vez na configuração inicial do banco. Em desenvolvimento, o `sync` sem `alter` permite verificar que os modelos estão compatíveis com as tabelas existentes.

Lição aprendida: Em projetos profissionais, é fundamental usar um sistema de **migrations** (como `sequelize-cli`) para versionar alterações no schema, em vez de depender do `sync()`.

16.4 Exportação CSV com Caracteres Especiais

Problema: O CSV gerado pelo sistema exibia caracteres acentuados corrompidos (ex: “Alimentação” virava “AlimentaÃ§Ã£o”) quando aberto no Microsoft Excel. O arquivo estava sendo gerado em UTF-8, mas o Excel no Windows não interpretava corretamente sem um indicador explícito de encoding.

Causa raiz: O Microsoft Excel, por padrão no Windows, espera arquivos CSV em `Windows-1252` (Latin-1). Ao abrir um arquivo UTF-8 sem BOM (Byte Order Mark), os caracteres acentuados e especiais são interpretados incorretamente.

Solução: Adicionar o BOM UTF-8 (`\ufeff`) no início do conteúdo CSV antes de enviá-lo na resposta. Além disso, o separador foi trocado de `,` para `;`, que é o padrão regional brasileiro no Excel (já que a vírgula é usada como separador decimal no formato `pt-BR`).

```
res.setHeader('Content-Type', 'text/csv; charset=utf-8');
res.setHeader('Content-Disposition', 'attachment; filename=despesas.csv');
res.send('\ufeff' + csvContent);
```

Lição aprendida: A internacionalização vai além da tradução de textos — formatos numéricos, separadores e encoding variam entre regiões e softwares.

16.5 Erro de Referência Circular no Eager Loading

Problema: Ao implementar o endpoint de fatura por cartão, a query Sequelize entrava em loop infinito e eventualmente esgotava a memória do Node.js. O erro ocorria porque a inclusão de `ContaCartao` nas despesas, que por sua vez incluía `Despesas`, criava uma referência circular no eager loading.

Causa raiz: As associações `ContaCartao.hasMany(Despesa)` e `Despesa.belongsTo(ContaCartao)` criavam um ciclo. Ao usar `include` recursivamente sem especificar a profundidade, o Sequelize tentava carregar infinitamente.

Solução: Definir includes explícitos e limitados em cada query, usando aliases (`as: 'conta'`, `as: 'despesas'`) e selecionando apenas os atributos necessários com `attributes: [...]`. Para o endpoint de faturas, a query foi reestruturada para buscar contas primeiro e depois despesas por conta, evitando joins profundos.

Lição aprendida: O eager loading do Sequelize é poderoso, mas deve ser usado com controle explícito. É preferível fazer múltiplas queries simples do que uma única query com includes profundos.

16.6 Tratamento de Tipos Decimais entre Sequelize e JavaScript

Problema: Os valores monetários retornados pela API apareciam como strings (`"1200.00"`) em vez de números (`1200.00`). Isso causava problemas nos cálculos do front-end, onde a soma de valores resultava em concatenação de strings ao invés de soma aritmética.

Causa raiz: O Sequelize retorna campos `DECIMAL(10,2)` como `string` por padrão para preservar a precisão (JavaScript `Number` é IEEE 754 float64 e pode perder precisão em operações decimais).

Solução: Nos services que realizam cálculos (dashboard, totais), os valores são explicitamente convertidos com `parseFloat()` antes das operações. O front-end também foi orientado a sempre usar `parseFloat()` ao consumir valores monetários da API.

Lição aprendida: A interface entre banco de dados (precisão fixa) e JavaScript (ponto flutuante) requer atenção constante. Para sistemas financeiros críticos, bibliotecas como `decimal.js` ou `big.js` são recomendadas.

17. Conclusão

O back-end do Sistema Financeiro Web foi construído seguindo **princípios sólidos de engenharia de software**: separação de responsabilidades, convenções consistentes, segurança em múltiplas camadas e uma API RESTful completa e documentada.

Pontos Fortes

- **Arquitetura limpa e escalável:** A separação em camadas permite que cada componente seja testado, mantido e evoluído independentemente.
- **Lógica de negócio robusta:** O parcelamento automático com transações ACID, o cálculo de dívidas futuras e o dashboard consolidado demonstram capacidade de implementar regras de negócio complexas.
- **Segurança:** Autenticação JWT, hash bcrypt, CORS restritivo, headers Helmet e conexão SSL com o banco de dados.
- **Documentação:** Swagger UI interativo com 45+ endpoints documentados, permitindo teste e exploração sem ferramentas externas.
- **Exportação de dados:** Suporte a CSV com formatação brasileira, BOM UTF-8 e separador adequado para Excel.

Evolução Futura

- **Testes automatizados:** Implementar suíte de testes unitários (Jest) para os services e testes de integração para os endpoints.
- **Migrations:** Adotar `sequelize-cli` para versionamento do schema do banco de dados.
- **Rate limiting:** Adicionar limitação de taxa de requisições para prevenção de abuso.
- **Notificações:** Sistema de alertas quando despesas ultrapassarem limites configurados.
- **Recorrência:** Agendamento automático de despesas e rendas recorrentes (mensais, semanais).
- **Multi-moeda:** Suporte a diferentes moedas com conversão automática.

Métricas do Projeto

Métrica	Valor
Endpoints da API	45+
Entidades do banco	7 tabelas
Services implementados	8
Controllers	8
Arquivos de rotas	9
Middlewares customizados	3
Dependências de produção	10
Linhas de documentação Swagger	450+

Documento gerado como parte da documentação técnica do projeto Sistema Financeiro Web.

Todos os endpoints foram validados e testados via Swagger UI e chamadas HTTP diretas.